Zeal Education Society's
ZEAL COLLEGE OF ENGINEERING & RESEARCH, PUNE – 41
(An Autonomous Institute Affiliated to Savitribai Phule Pune University)
NAAC Accredited with A+ Grade / ISO 21001:2018

DEPARTMENT OF ELECTRONICS AND COMPUTER ENGINEERING

Experiment No.: 10

Interfacing LDR and MQ3 sensor with STM32F4xx

Name: _____

Roll No.: _____ *Batch:* _____

Date of Performance: _____

Date of Assessment: _____

Particulars	Marks
Marks for Regularity (05)	
Marks for Performance (15)	
Understanding (Viva) (05)	
Total (25)	
Signature of Course Teacher	

Experiment: 10

AIM: Interfacing LDR and MQ3 sensor with STM32F4xx.

OBJECTIVE:

1. To interface the **LDR (Light Dependent Resistor)** and **MQ-3 alcohol/gas sensor** with the STM32F4xx microcontroller. To understand the working principle of the HC-SR04.
2. To understand the working principle of the LDR — a **photo resistive sensor** whose resistance decreases with increasing light intensity

APPARATUS:

1. STM32F407 module development board.
2. LDR module and MQ3 Sensor module.
3. Connecting wire.
4. Adapter.

THEORY:

A Light Dependent Resistor (LDR) is a type of passive electronic sensor used to detect light.

It's made up of two conductors separated by an insulator which becomes more conducting when exposed to high levels of light intensity, forming a variable resistor in the circuit.

This allows it to measure the amount and brightness or darkness within its environment and provide information accordingly.

LDR light sensors are typically used as part of automated lighting control systems for energy conservation purposes such as dimming lights based on natural daylight that can be detected outside without human intervention.

Furthermore, they have found the applications of sensor in motion detectors like burglar alarm security systems because they can tell if something passes through their beamline quickly enough from dark-to-light

conditions created with most forms of ambient lighting sources present today such as LED, etc., making them ideal sensing tools both commercially & domestic level settings alike.

An LDR, or Light Dependent Resistor is a type of optoelectronic sensor that's used for detecting light in the environment.

LDR works by sensing changes in illumination and converting them into electrical signals that can be read off from its two terminals.

This makes it an essential tool in many automation systems because it helps detect objects without any physical contact with them.

In addition to this, they have low power requirements making them ideal for energy-saving applications like solar cells and garden lighting systems where their high sensitivity allows precise illumination control even under tightly controlled conditions.

Their small size further increases flexibility as well giving these devices versatility across multiple different projects while still being able to fit inside tight spaces too!

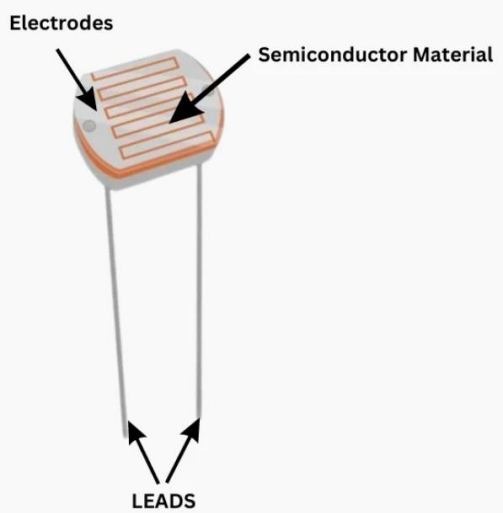
Most home appliances, outdoor lighting, and streetlights are normally operated and maintained manually.

This is not only dangerous, but it also wastes energy due to staff negligence or unforeseeable events when turning this electrical equipment ON and OFF. Therefore, by using a light sensor, we can automatically shut off the loads based on the intensity of the daylight.

By sensing the radiant energy present in a relatively small range of frequencies often referred to as "light," which runs in frequency from the "Infra-red" to "Visible" up to "Ultraviolet" light spectrum , a light sensor produces an output signal that indicates the intensity of light.

The LDR light sensor is a passive component that produces an electrical signal from this "light energy," whether it is in the visible or infrared portions of the spectrum.

Because they transform light energy (photons) into electricity, light sensors are more generally referred to as "Photoelectric Devices" or "Photo Sensors" (electrons).



The photoconductivity theory underlies the operation of this resistor. It is nothing more than the fact that when light strikes its surface, the material's conductivity decreases and the electrons in the device's valence band are stimulated to the conduction band.

These incident light photons must have energy larger than the semiconductor material's band gap. Consequently, the electrons quickly move from the valence band to the conduction band.

Applications of LDR:

- The photoresistor is typically used to measure light intensity and presence.
- These sensors are used in lights that turn on and off automatically based on light
- Smoke Detector Alarm, Automatic Lighting Clock
- These sensors are used in the design of optical circuits
- Proximity switch for photos
- Security measures utilizing lasers
- In Solar Street lighting it can be used
- Light meters for cameras
- It can be used in Radio clocks

MQ3 Sensors:

The MQ3 gas sensor module is excellent for detecting gas leaks (in home and industry). It is capable of detecting CO, CH4, Benzine, Hexane, LPG, and Alcohol.

Measurements may be made as soon as feasible due to their high sensitivity and quick reaction rate. The potentiometer can be used to modify the sensor's sensitivity.

This low-cost semiconductor gas sensor module is very simple to operate and has both analog and digital output. This Gas Sensor module can be used in breathalyzers because it is sensitive to alcohol. Benzine is similarly only weakly sensitive to MQ3 Sensor.

Features of MQ3 sensor:

- You can easily add this sensor to circuits because it has a simple SIP header interface.
- The MQ-3 Gas Sensor detects alcohol vapor accurately with semiconductor technology.
- The MQ-3 sensor is flexible and may be simply attached to a variety of microcontrollers.
- With its great sensitivity and rapid reaction, the sensor detects alcohol vapor quickly enough to take action.

1. Using the STM32CubeIDE.

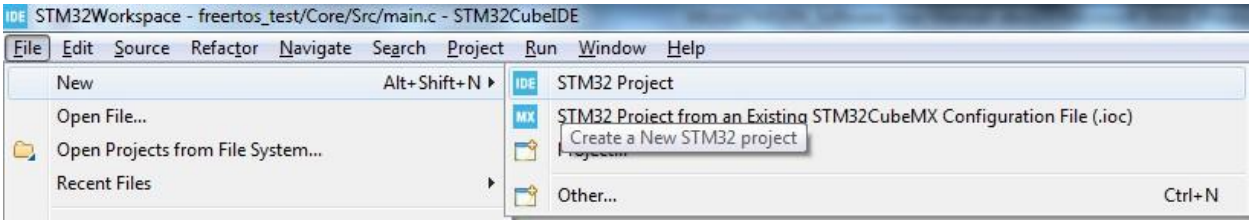
1.1. Creating/Choosing a Workspace.

When you open the STM32CubeIDE for the first time, you will be directed to create a workspace for your projects. You can choose the default location for the project provided by the IDE (which will generally be located in the **users** folder in C: drive) or you can assign a different folder for the workspace. All your projects will be stored and referenced from this workspace folder. STM32CubeIDE is a eclipse based IDE and so you can have multiple projects in your projects window. Only one project can be active at one time. The Active project will get compiled or debugged or downloaded when you use the shortcut keys for the Compile, Debug or RUN actions.

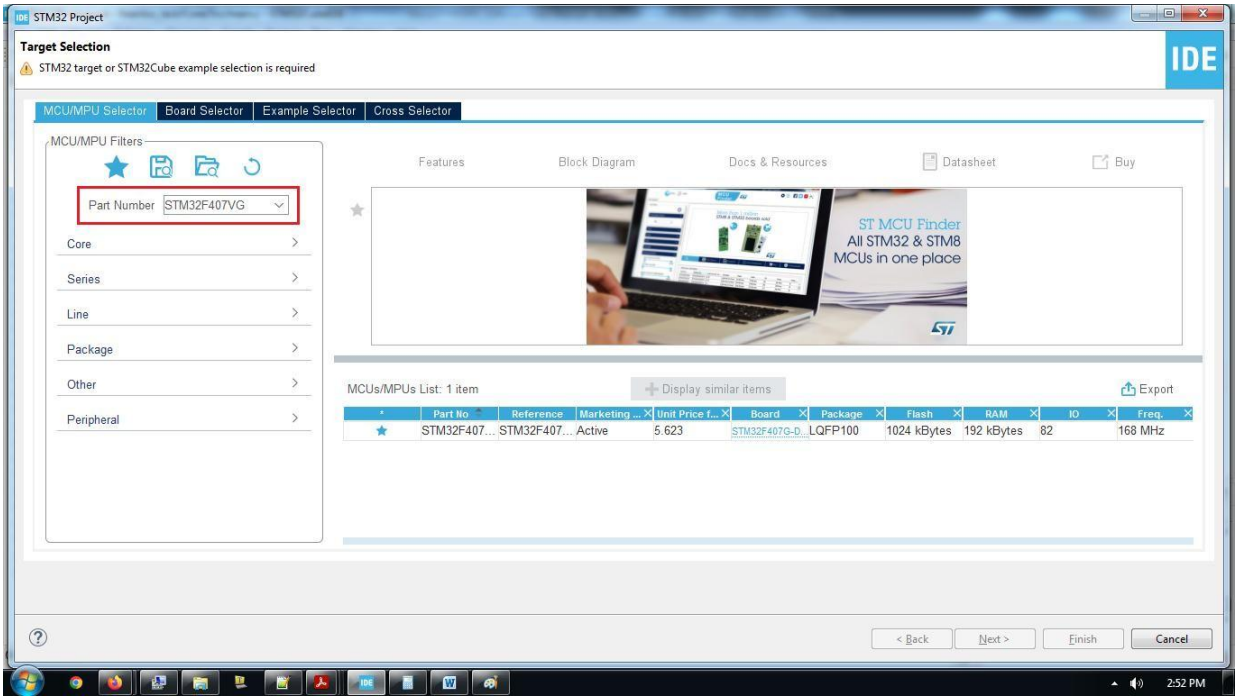
1.2. Creating a New Project.

Open the STM32CubeIDE (preferably Run as administrator)

Step 1: To create a new project, click on File→New→STM32 Project.



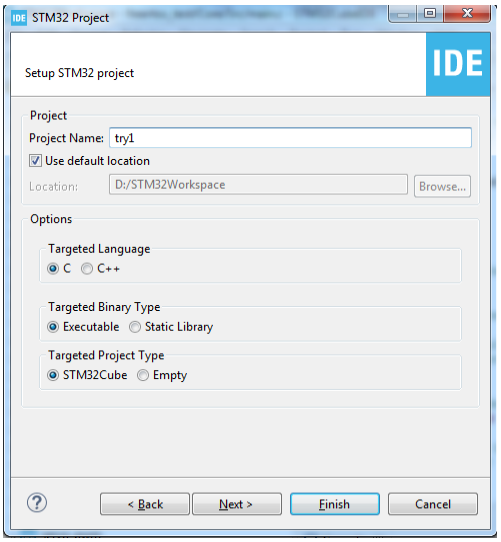
Step 2: Select the MCU part number.



In the Target Selection window, enter the part number **STM32F407VG** (this part is populated on the board), in the Part Number window.

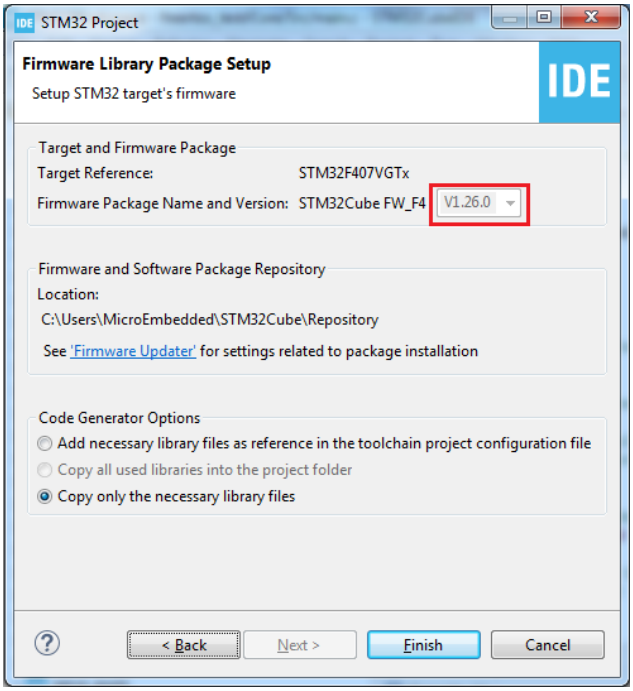
MCU/MPU Selector → Part Number.

Now select the part (click near the star) and click Next. Step 3:

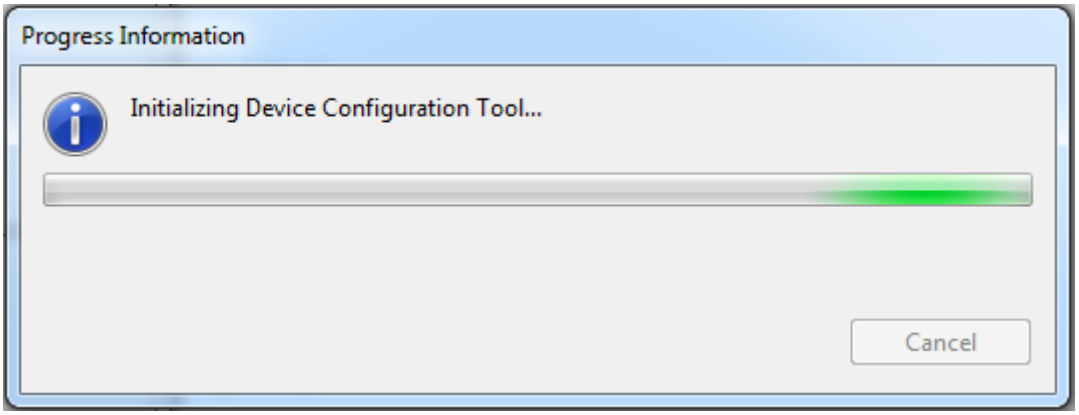


In the Project Name window, give a unique name to the project. (you cannot have two projects with the same name). Click Next.

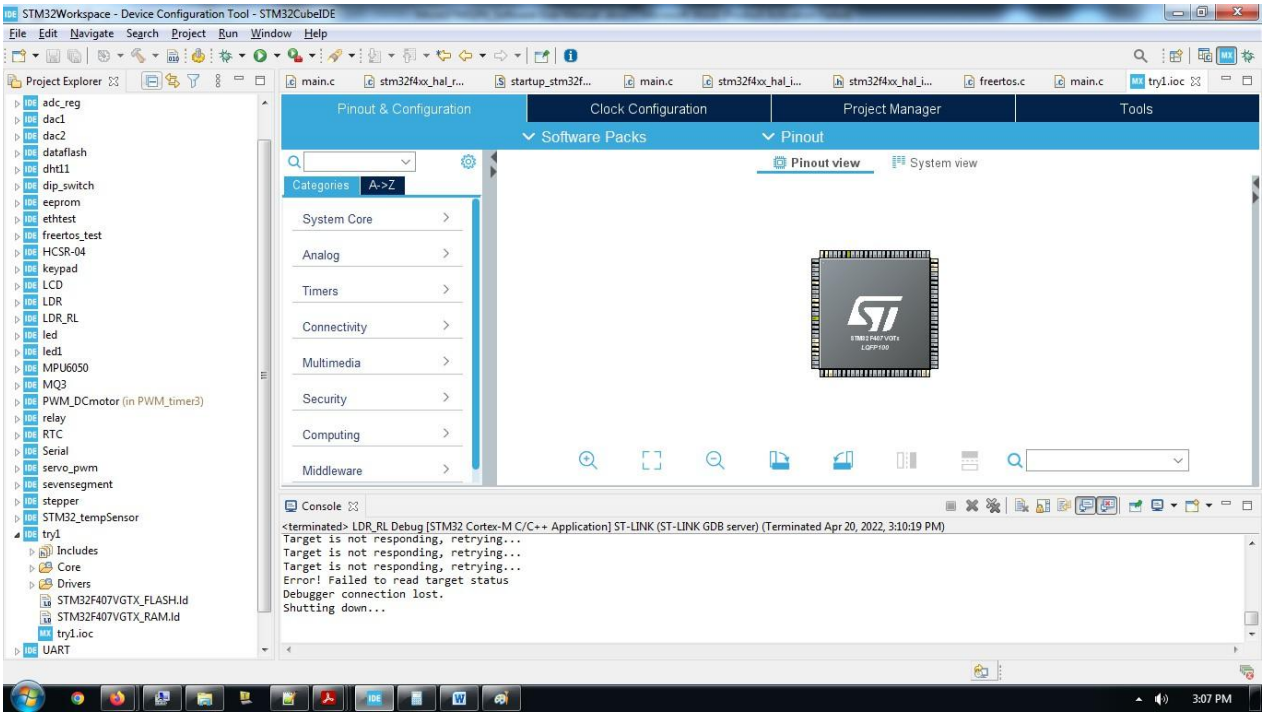
Step 4:



Make sure that the Firmware Package Name and Version is **V1.26.0.**
Click Finish to complete the process of creating the new project.



After the process is complete the project Template will get created.

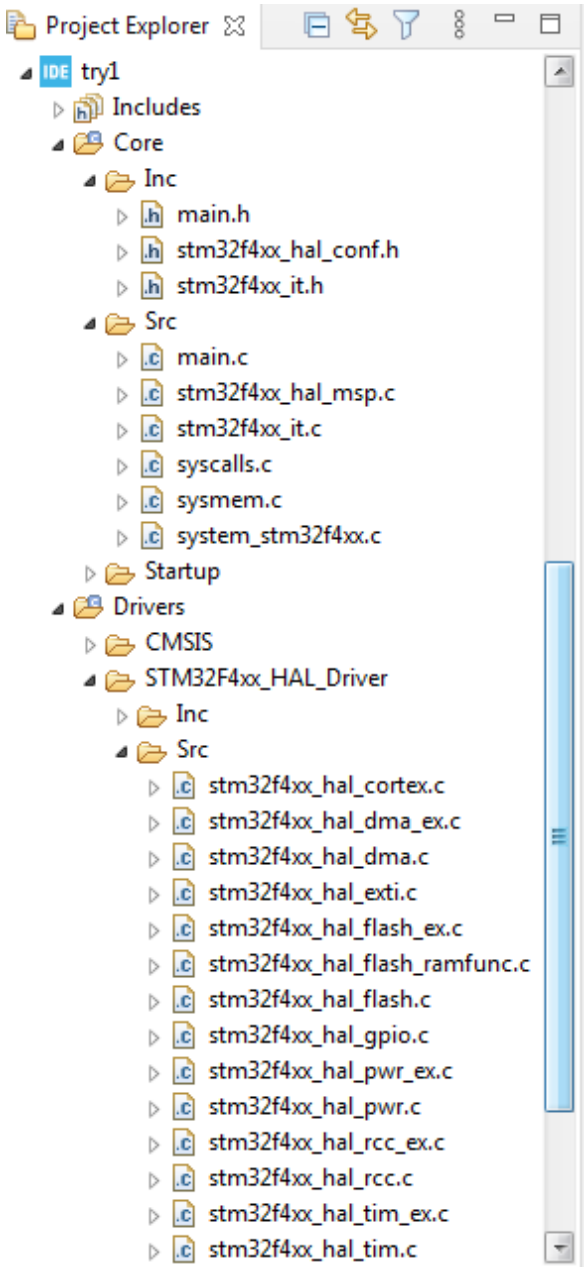


The Project Structure is as follows:
All the source code (.c) and header (.h) files for the HAL (library function for peripherals) are located in
C files :Project_name →Drivers→STM32F4xx_HAL_Driver→Src
.h files :Project_name →Drivers→STM32F4xx_HAL_Driver→Inc

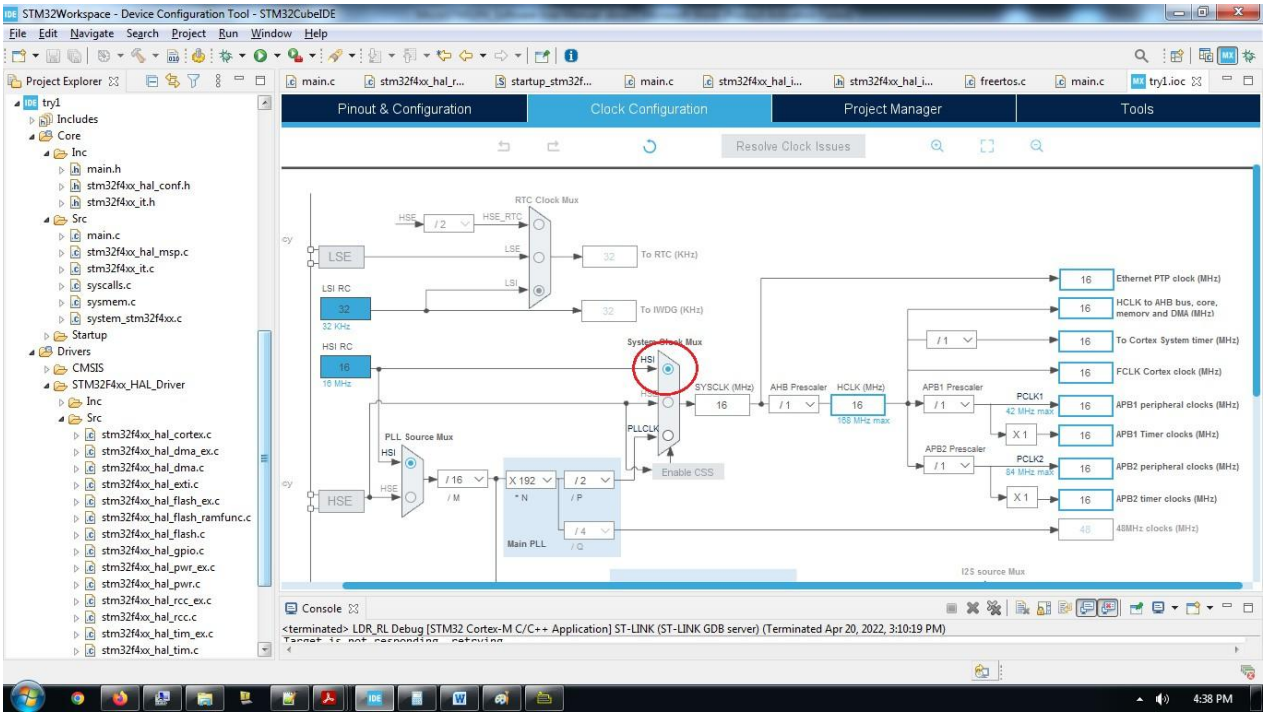
All the source code (.c) and header (.h) files for the user program are located in C files :Project_name →Core→Src
.h files :Project_name →Core→Inc

The user can write the program in main.c file.

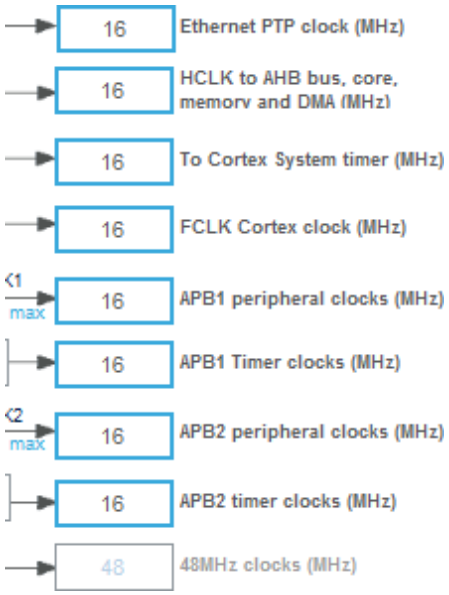
The code for clock initialization and HAL initialization are already done and available in main.c file.



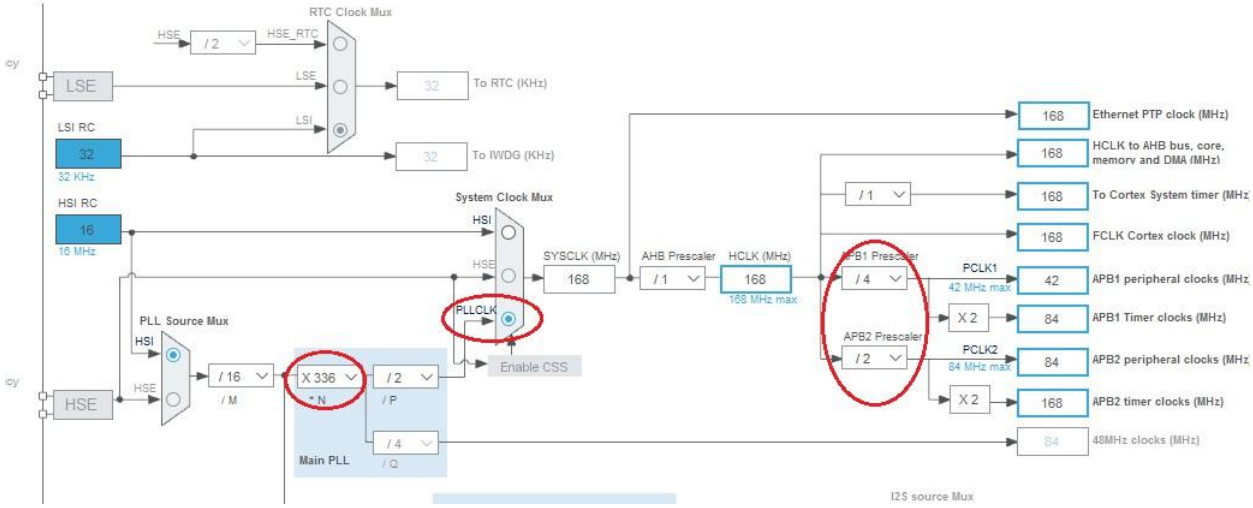
1.3. Setting the Clock Configuration.



Users can use the default clock settings for 16Mhz if they do not wish to make changes. Here we are using the (HSI) internal High Frequency RC clock to generate the clock signals for all the peripherals like AHB bus, APB bus etc.



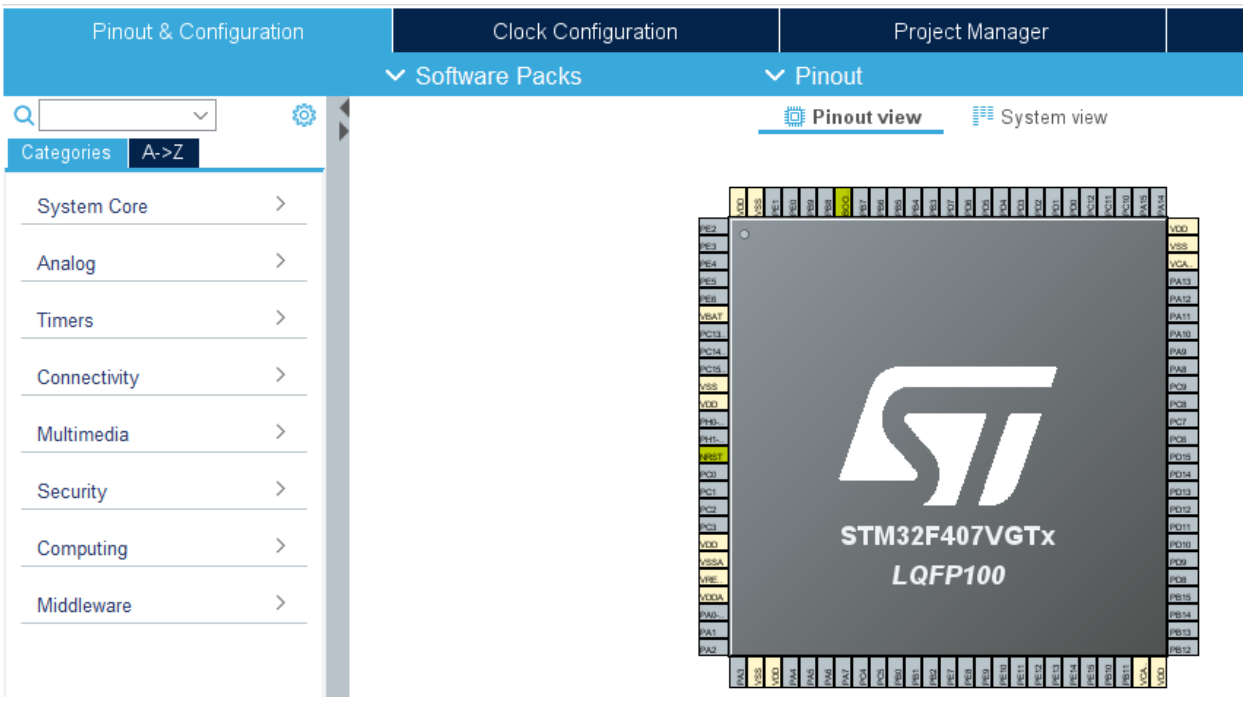
If the user wants the maximum clock frequency of 168 MHz, the PLL should be set as follows;



In the PLL section make N = 336
In System Clock Mux select PLLCLK In
APB1 Prescalar use /4
In APB2 Prescalar use /2

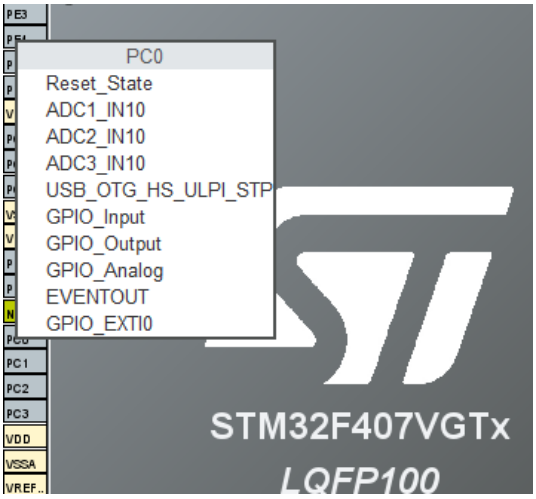
This will set all the clocks to the core and peripherals for maximum allowed frequency.

1.4. Pinout & Configuration.

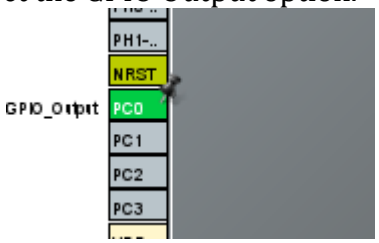


We can use the GUI tool in the IDE to set the pin configuration.

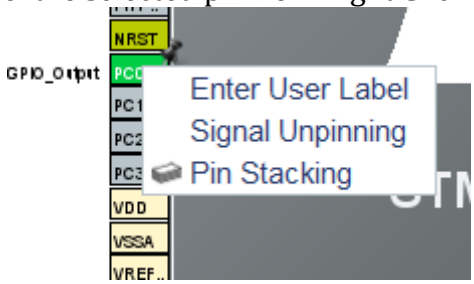
Select the pin (let's take pin PC0 which is connected to LED1). Locate the pin on the MCU pin map. Left Click on the pin PC0.



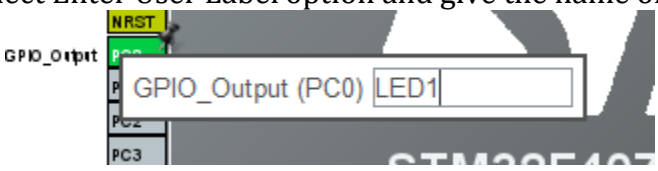
Select the GPIO Output option.



To label the selected pin now Right Click on the pin

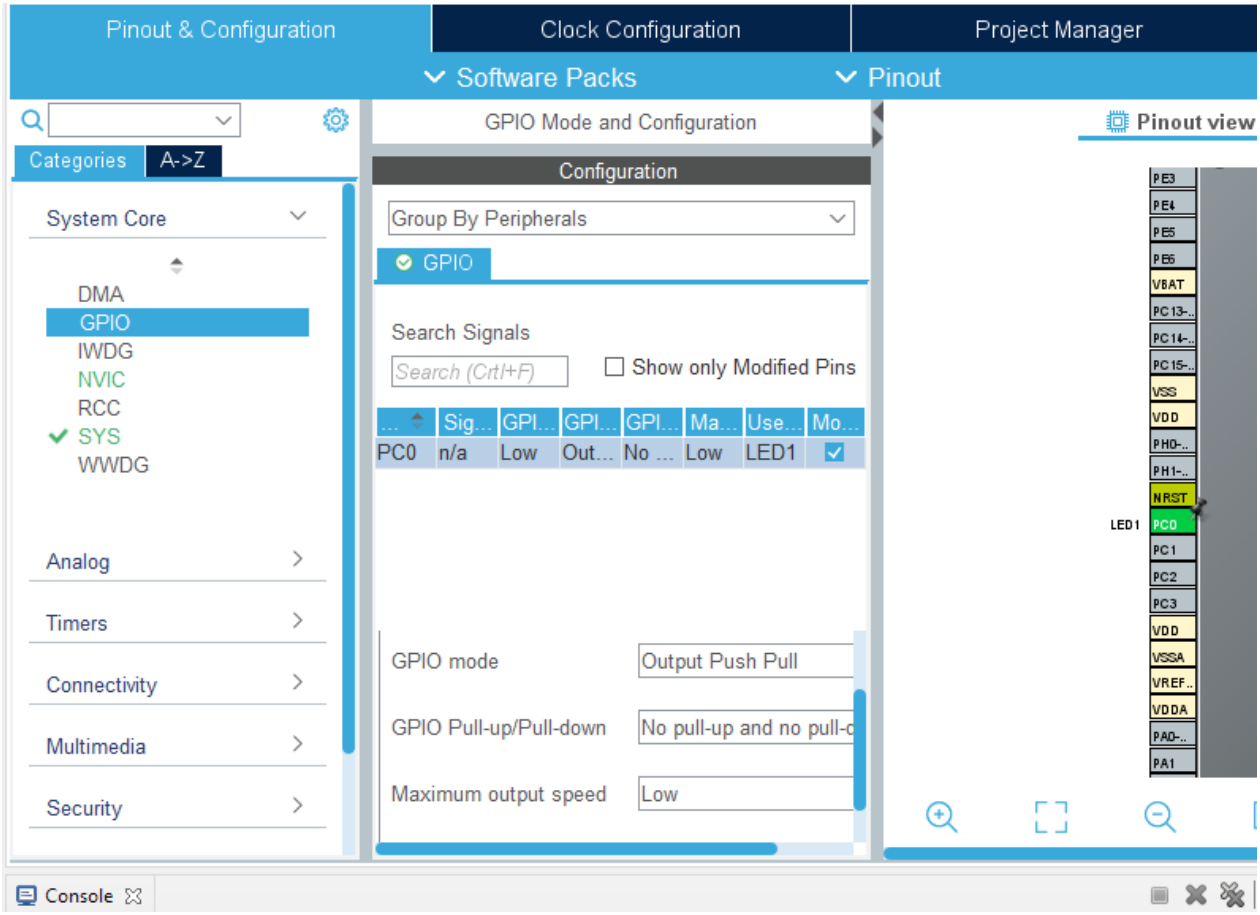


Select Enter User Label option and give the name of your choice in the window



Now when you press enter, the pin name will be set to LED1. This name will be used by the library and automatic code generation tool throughout the code.

You can also set the pin parameters from the tool.
Expand the System Core and select GPIO. The selected GPIO will be shown and below you can set the parameters for the selected Pin.



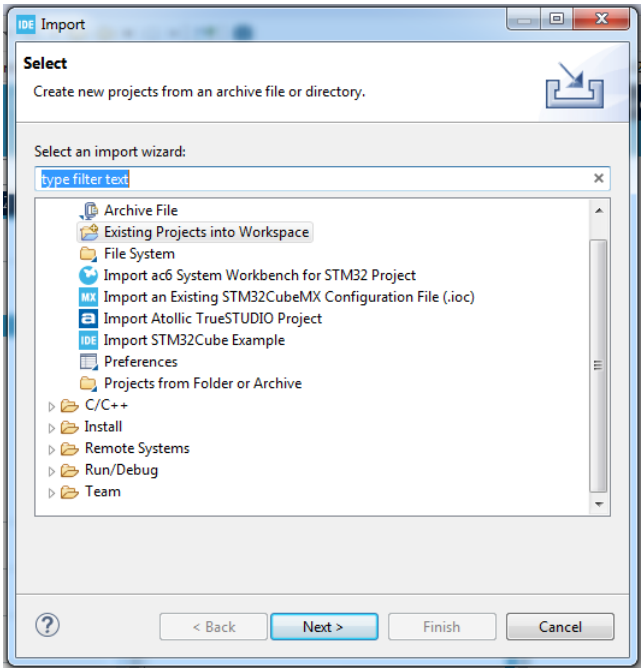
The same procedure applies to multiple pins and peripherals. To set the parameters for the peripherals, set the pins, select the peripherals from the Pinout &Configuration settings block and make appropriate changes.

After you have set the parameters, save the project. You will be prompted to generate code. When you click yes, the IDE will generate the initialization code and template in the main.c file.

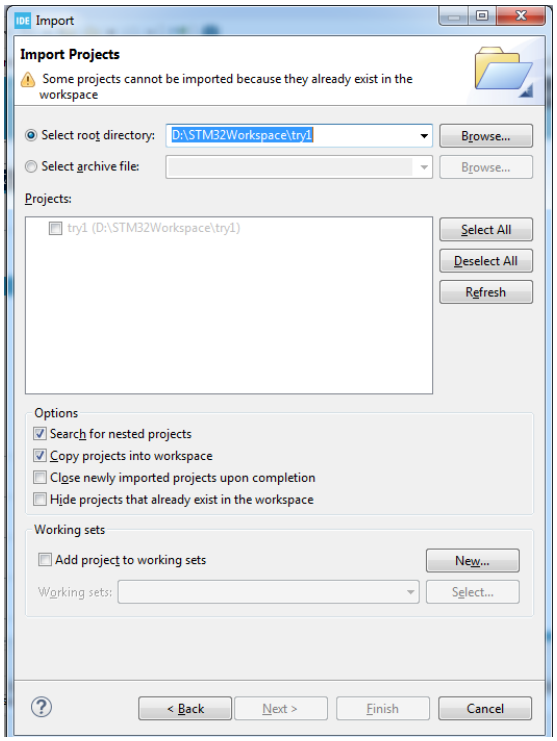
Alternatively, you can also click on the  Device Configuration Tool Code Generation to generate the code. Or go to Project →Generate Code.

1.5. Importing an existing project in to the workspace.

When you want to import an existing project in to the workspace, click on File → Import.



Select – Existing Projects into Workspace. Click Next.



In select root directory, browse to the folder which contains the project to be imported. Also make sure that the Copy projects into workspace field is selected in the options menu. Click Finish. The project will be imported into the workspace.

1.6. Building / Compiling the project.

After writing the code in the main.c file compile the project by Right clicking on the project name and click Build Project.

Also you can select the project name. click on Project →Build Project.

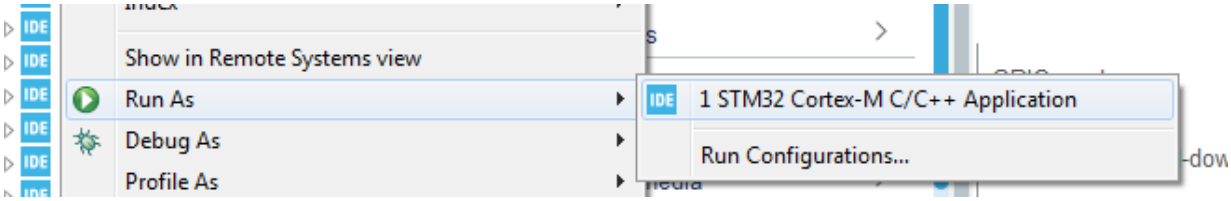
Else you can select the project and click on  icon to build the project.

After successful build the bin file and elf file will be created. This file can be flashed into the microcontroller using the ST emulator.

1.7. Downloading into the Microcontroller.

To download the hex file into the microcontroller, connect the Emulator to PC using the USB cable and the FRC cable to connector CN22 (JTAG Header).
Power on the board.
Make sure that Boot0 and Boot1 switches are towards GND (2-3). And
switch SW13 is towards 2-3.

Now select the project, right click and select **Run As→ STM32 cortex m C/C++ Application**



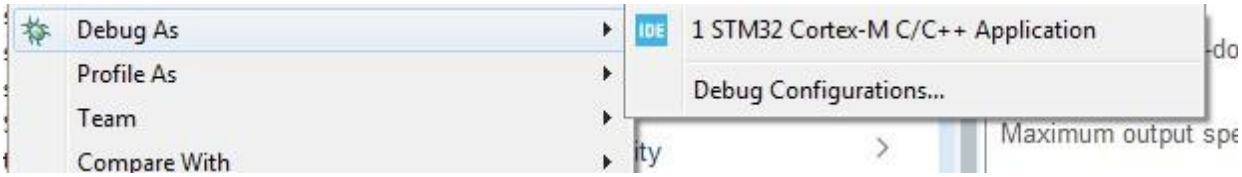
Also, you can select the project →Run→Run. Else

click on  icon to download the file.

Press RESET switch SW38 and observe the output.

1.8. Debug the code.

To debug the code; select the project →Right click →Debug As →STM32 Cortex-M C/C++ Application.



Also select the project →Run →Debug.

Or select the project →click on debug icon. 

The hex fie will get loaded and the program will halt at main function. The IDE will enter the debug perspective and you can use the step into to step out to debug the code line by line.

CONCLUSION: